

STEP 1 — Dataset

```
documents = [
    "The flight was delayed due to weather",
    "The airplane was late because of rain",
    "Customer service was very bad",
    "The support staff was extremely poor",
    "I loved the movie and the acting",
    "The film was amazing and enjoyable",
    "This product is very good",
    "This item is excellent",
    "I hate this service",
    "The experience was terrible",
    "The food was tasty and fresh",
    "The meal was delicious",
    "He is reading a book",
    "She is studying for exams",
    "The sky is blue today"
]
```

STEP 2 — Preprocessing

```
import nltk
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('stopwords')

stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = text.lower()
    text = text.translate(str.maketrans("", "", string.punctuation))
    tokens = word_tokenize(text)
    tokens = [w for w in tokens if w not in stop_words and len(w) > 2]

    return tokens
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
True
```

```
processed_docs = [preprocess(doc) for doc in documents]
```

```
processed_docs = [preprocess(doc) for doc in documents]
processed_docs
```

```
[[['flight', 'delayed', 'due', 'weather'],
  ['airplane', 'late', 'rain'],
  ['customer', 'service', 'bad'],
  ['support', 'staff', 'extremely', 'poor'],
  ['loved', 'movie', 'acting'],
  ['film', 'amazing', 'enjoyable'],
  ['product', 'good'],
  ['item', 'excellent'],
  ['hate', 'service'],
  ['experience', 'terrible'],
  ['food', 'tasty', 'fresh'],
  ['meal', 'delicious'],
  ['reading', 'book'],
  ['studying', 'exams'],
  ['sky', 'blue', 'today']]]
```

STEP-3 — Cosine Similarity using TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
processed_texts = [" ".join(doc) for doc in processed_docs]
```

```
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(processed_texts)
```

```
cosine_sim = cosine_similarity(tfidf_matrix)
```

```
cosine_sim
```

```
array([[1.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 1.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 1.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.34306349, 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 1.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 0.      , 1.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 0.      , 0.      ,
        1.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 1.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 1.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.34306349, 0.      , 0.      ,
        0.      , 0.      , 0.      , 1.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 1.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ,
        1.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 1.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 1.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 1.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ],
       [0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 0.      ,
        0.      , 0.      , 0.      , 0.      , 1.      ]])
```

```
pairs = []

for i in range(len(documents)):
    for j in range(i+1, len(documents)):
        pairs.append((i, j, cosine_sim[i][j]))

top5 = sorted(pairs, key=lambda x: x[2], reverse=True)[:5]

for i, j, score in top5:
    print(f"Doc {i} & Doc {j} → Cosine Similarity = {score:.2f}")
```

```
Doc 2 & Doc 8 → Cosine Similarity = 0.34
Doc 0 & Doc 1 → Cosine Similarity = 0.00
Doc 0 & Doc 2 → Cosine Similarity = 0.00
Doc 0 & Doc 3 → Cosine Similarity = 0.00
Doc 0 & Doc 4 → Cosine Similarity = 0.00
```

STEP-4: Jaccard Similarity

```
def jaccard_similarity(doc1, doc2):
    return len(set(doc1) & set(doc2)) / len(set(doc1) | set(doc2))
```

```
for i, j, _ in top5:
    print(f"Doc {i} & Doc {j} → Jaccard = {jaccard_similarity(processed_docs[i], processed_docs[j]):.2f}")
```

```
Doc 2 & Doc 8 → Jaccard = 0.25
Doc 0 & Doc 1 → Jaccard = 0.00
Doc 0 & Doc 2 → Jaccard = 0.00
Doc 0 & Doc 3 → Jaccard = 0.00
Doc 0 & Doc 4 → Jaccard = 0.00
```

STEP-5: WordNet Semantic Similarity

```
from nltk.corpus import wordnet as wn
nltk.download('wordnet')
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
True
```

```
def wordnet_similarity(word1, word2):
    syn1 = wn.synsets(word1)
    syn2 = wn.synsets(word2)
    if syn1 and syn2:
        return syn1[0].wup_similarity(syn2[0])
    return 0
```

```
def sentence_wordnet_similarity(doc1, doc2):
    scores = []
    for w1 in doc1:
        for w2 in doc2:
            score = wordnet_similarity(w1, w2)
            if score is not None:
                scores.append(score)
    if scores:
        return sum(scores) / len(scores)
    return 0
```

```
pairs_to_test = [
    (0, 1), # flight delayed ↔ airplane late
    (2, 3), # bad service ↔ poor support
    (4, 5), # movie ↔ film
    (6, 7), # product good ↔ item excellent
    (8, 9), # hate service ↔ terrible experience
    (10, 11), # tasty food ↔ delicious meal
    (12, 13), # reading ↔ studying
    (0, 5),
    (2, 8),
    (1, 9)
]

for i, j in pairs_to_test:
    score = sentence_wordnet_similarity(processed_docs[i], processed_docs[j])
    print(f"Doc {i} & Doc {j} → WordNet Similarity = {score:.2f}")
```

```
Doc 0 & Doc 1 → WordNet Similarity = 0.21
Doc 2 & Doc 3 → WordNet Similarity = 0.29
Doc 4 & Doc 5 → WordNet Similarity = 0.37
Doc 6 & Doc 7 → WordNet Similarity = 0.23
Doc 8 & Doc 9 → WordNet Similarity = 0.27
Doc 10 & Doc 11 → WordNet Similarity = 0.32
Doc 12 & Doc 13 → WordNet Similarity = 0.39
Doc 0 & Doc 5 → WordNet Similarity = 0.21
Doc 2 & Doc 8 → WordNet Similarity = 0.39
Doc 1 & Doc 9 → WordNet Similarity = 0.20
```

STEP 6: Comparison of Similarity Measures

Cosine similarity detected copied or slightly modified documents effectively because it uses TF-IDF to capture important shared terms between documents. However, cosine similarity failed when documents were paraphrased using different words with the same meaning, resulting in low similarity scores. Jaccard similarity worked well only when there was a high overlap of exact words, but it failed for paraphrased sentences since it does not consider semantic meaning. WordNet semantic similarity successfully identified meaning similarity between sentences that used synonyms such as flight and airplane or movie and film. WordNet performed better in detecting paraphrased text compared to lexical methods. Cosine similarity sometimes produced false positives due to common words, while WordNet reduced false positives by focusing on semantic relationships. Overall, lexical similarity measures work on surface text, whereas semantic similarity captures deeper meaning. Combining lexical and semantic methods provides more accurate text similarity results.

STEP 7: Lab Report Objective

The objective of this experiment is to understand and compare different text similarity measures in Natural Language Processing. The experiment focuses on computing similarity between text documents using Cosine similarity, Jaccard similarity, and WordNet-based semantic similarity, and analyzing their behavior on copied, paraphrased, and unrelated texts.

Dataset Description

A dataset of 15 text documents was manually created for this experiment. The dataset includes original sentences, slightly modified copies, paraphrased sentences using synonyms, and completely unrelated texts. This allows effective comparison of lexical and semantic similarity measures. The dataset was designed to clearly observe how different similarity methods behave for different types of text.

Preprocessing Steps

The text data was preprocessed before similarity computation. All documents were converted to lowercase to maintain uniformity. Punctuation marks were removed to reduce noise. Stopwords were removed to keep only meaningful words. Tokenization was performed to split sentences into individual words. These preprocessing steps help improve the accuracy of similarity calculations.

Cosine Similarity Results

Cosine similarity was computed using TF-IDF feature representation. The results showed high similarity scores for documents that shared common important words or were slightly modified copies. However, cosine similarity produced low scores for paraphrased sentences that used different words with similar meaning. This shows that cosine similarity is effective for lexical similarity but limited in capturing semantic meaning.

Jaccard Similarity Results

Jaccard similarity was calculated using word sets derived from the documents. It worked well when documents had high word overlap. However, it failed for paraphrased sentences because it depends only on exact word matching. The results demonstrate that Jaccard similarity is suitable only for detecting copied text with common vocabulary.

WordNet Semantic Similarity Results

WordNet-based semantic similarity was used to measure meaning similarity between words and sentences. It successfully identified similarity between sentences using synonyms such as flight and airplane or movie and film. WordNet similarity performed better than cosine and Jaccard for paraphrased texts, proving its effectiveness in capturing semantic relationships.